

# Wo wir stehen

1

**Termin 1: verstanden:** Was ein Agent ist, was mit Ihren Daten passiert, welche vier Bausteine den Unterschied machen.

2

**Termin 2: gebaut:** Ihr erster eigener Skill, und ein erster Schritt an Ihrem echten Fall.

3

**Heute: verknüpfen:** Wissen, das zusammenhängt. Zusammenarbeit über Instanzen. Und die Frage, was es wirklich spart.

**Und Ihre Fehlerliste?** Wer aufgeschrieben hat, wo der eigene Skill danebenlag, hebt sie auf. Wir brauchen sie in Block 7.

# Wie Ihr Wissen zur KI kommt

Das Modell kennt Ihr Haus nicht. Sie entscheiden, was es erfährt und auf welchem Weg.



# Was das Modell weiß, und was nicht

## AUS DEM TRAINING

### Allgemeines

- Sprache, Fachwissen, Zusammenhänge der Welt
- gelernt bis zu einem Stichtag
- nichts davon lässt sich nachträglich ändern

## NIE GESEHEN

### Alles von Ihnen

- Ihre Dateien, Ihre Abläufe, Ihr Haus
- alles, was nach dem Stichtag passiert ist
- auch das Gespräch von gestern

## DER SCHREIBTISCH

### Was Sie hinlegen

- alles, was in dieser Sitzung sichtbar ist
- begrenzt: der Tisch hat eine Größe
- am Anfang und am Ende liest es am besten

**Daraus folgt alles Weitere:** Was Ihr Haus betrifft, müssen Sie hineingeben. Die einzige offene Frage ist, auf welchem Weg.

Der Schreibtisch heißt im Fachjargon Kontextfenster. Dass ein größerer Tisch nicht automatisch besser ist, ist untersucht: In der Mitte langer Eingaben übersehen Modelle mehr.

Quellen: [1], [2] (Verzeichnis im Anhang)

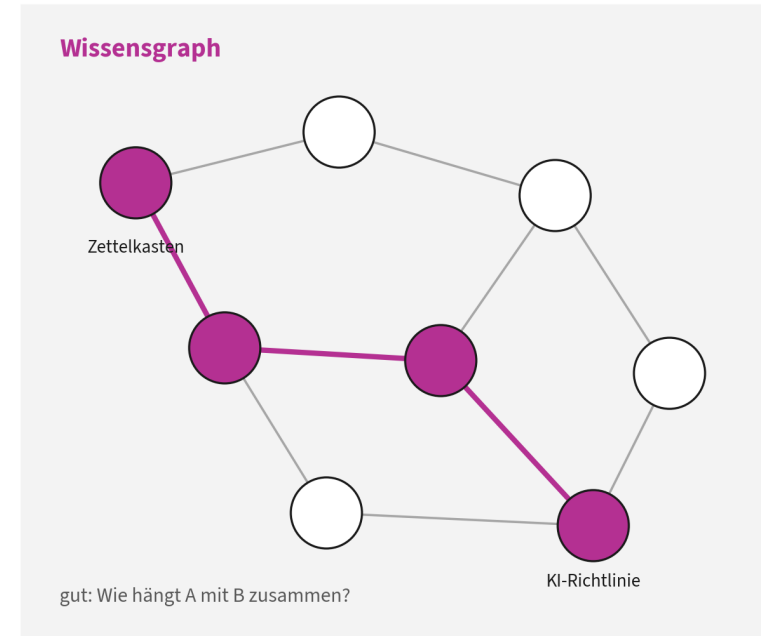
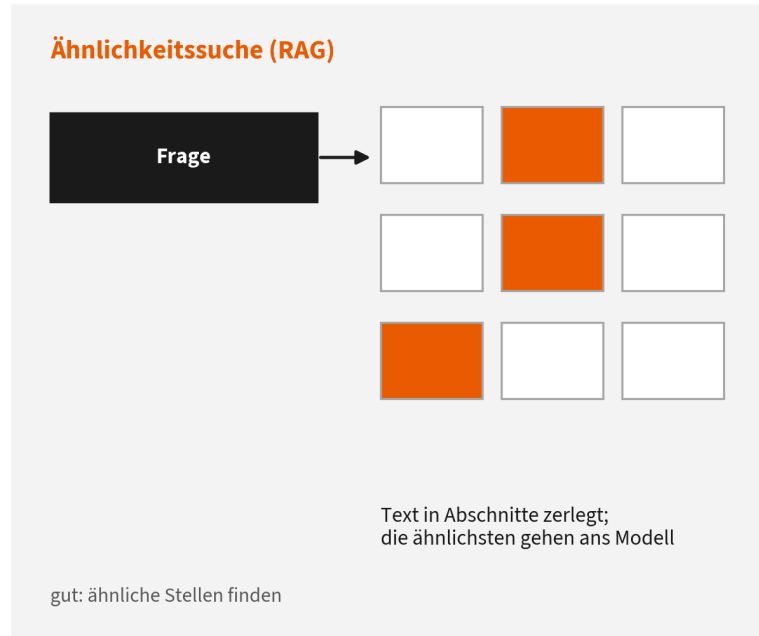
# Fünf Wege, Wissen hineinzugeben

|   | WEG                          | WAS PASSIERT                                   | WOFÜR ES SICH LOHNT                                            |
|---|------------------------------|------------------------------------------------|----------------------------------------------------------------|
| 1 | <b>Hineinkopieren</b>        | Text in die Anfrage einfügen                   | für eine einzelne Frage, sofort, ohne Einrichtung              |
| 2 | <b>Ordner zeigen</b>         | Der Agent liest die Dateien selbst             | für ein Projekt, an dem Sie gerade arbeiten                    |
| 3 | <b>Dauerhaft hinterlegen</b> | Regeldatei und Skills, wie in Block 2          | für alles, was immer gilt, unabhängig vom Projekt              |
| 4 | <b>Durchsuchbar machen</b>   | Passende Stellen werden gesucht und mitgegeben | für viele Dokumente, aus denen nur wenig gebraucht wird        |
| 5 | <b>Verknüpfen</b>            | Ein Netz aus Begriffen und ihren Beziehungen   | für Fragen, deren Antwort über mehrere Dokumente verstreut ist |

**Die ersten drei kennen Sie schon.** Weg 4 heißt im Fachjargon RAG, Weg 5 Wissensgraph. Um die beiden geht es jetzt.

Quellen: [3], [4], [5] (Verzeichnis im Anhang)

# Ähnlichkeit oder Beziehung?



**RAG findet ähnliche Stellen,** ein Graph erklärt Zusammenhänge. Für Fragen über ganze Sammlungen liefert die Graph-Variante umfassendere Antworten.

Quellen: [3], [6], [2] (Verzeichnis im Anhang)

# Was nehme ich wofür?

| Ihre Lage                              | Der passende Weg          | Warum                                       |
|----------------------------------------|---------------------------|---------------------------------------------|
| Eine Frage zu einem Text               | Hineinkopieren            | Alles andere ist Aufwand ohne Gegenwert     |
| Ein Projekt, an dem Sie sitzen         | Ordner zeigen             | Der Agent sucht sich selbst, was er braucht |
| Etwas, das immer gilt                  | Regeldatei oder Skill     | Steht einmal da und wirkt in jeder Sitzung  |
| Viele Dokumente, wenig gebraucht       | Durchsuchbar machen (RAG) | Nur die passenden Stellen kosten Platz      |
| Antwort über mehrere Quellen verstreut | Wissensgraph              | Die Verbindung ist die Antwort              |

**Wann es sich nicht lohnt:** Unter etwa fünfzig Dokumenten ist Hineinkopieren fast immer schneller als jede Einrichtung. Und ein Graph zahlt sich erst aus, wenn Sie wirklich Fragen über Zusammenhänge stellen und nicht nur nachschlagen.

Quellen: [3], [6] (Verzeichnis im Anhang)

# Die Idee ist älter als die Technik

1981

## Luhmann

- Zettelkasten mit rund 90.000 Notizen
- feste Adressen, Verweise
- der Kasten als Gesprächspartner

2022

## Forte

- Building a Second Brain
- erfassen, ordnen, verdichten, nutzen
- Projekte statt Ordnerlogik

HEUTE

## Sprachmodelle

- verdichten und verknüpfen automatisch
- ein Netz statt einer Ordnerstruktur
- Fragen beantworten mit Pfaden

**Eine Notiz ist erst wertvoll**, wenn sie mit einer anderen verbunden ist. Neu ist nicht die Idee, neu ist, dass das Verbinden jemand anderes erledigt.

Quellen: [7], [8], [9] (Verzeichnis im Anhang)

# Graphify oder Graft?

|          | Graphify                              | Graft                             |
|----------|---------------------------------------|-----------------------------------|
| Bild     | Landkarte zum Aufschlagen und Teilen  | Navi, das im Hintergrund mitläuft |
| Material | Code, Texte, PDFs, Bilder, Medien     | nur Code                          |
| Nutzung  | auf Anfrage: query, path, explain     | automatisch bei jedem Prompt      |
| Im Team  | Graph wird ins Git eingchecked        | Graph bleibt lokal                |
| Stärke   | Second Brain, Literatur, Lehrmaterial | große Software-Projekte           |

**Faustregel:** Wer mit Papern und Unterlagen arbeitet, beginnt mit Graphify. Wer Forschungssoftware entwickelt, nimmt Graft dazu.

Quellen: [10], [11] (Verzeichnis im Anhang)

# Der Wissensgraph in Aktion

LIVE

ca. 6 Minuten

1. /graphify second-brain-demo/unterlagen (vorher gebaut)
1. graph.html: Themengruppen und Verbindungen
1. GRAPH\_REPORT.md: zentrale Begriffe
1. /graphify path „Zettelkasten“ „KI-Richtlinie“
1. optional: graft viz im Code-Beispiel

## Falls es hakt

Graph liegt vorab gebaut im Repo.  
Screenshots im Dozentenordner.

**Beobachtungsauftrag:** Woher stammt jede Aussage: steht sie in der Quelle, oder wurde sie erschlossen?

Quellen: [10], [11] (Verzeichnis im Anhang)

# Zu mehreren am selben Stand arbeiten

Damit nicht jede und jeder im Team demselben Assistenten dieselben Dinge neu erklären muss.



# Jede Instanz startet bei null

## OHNE GEMEINSAMEN KONTEXT

### Fünf Clauses, fünf Wahrheiten

- widersprüchliche Texte und Begriffe
- doppelte Arbeit
- längst getroffene Entscheidungen werden neu diskutiert

## MIT GEMEINSAMEM REPOSITORY

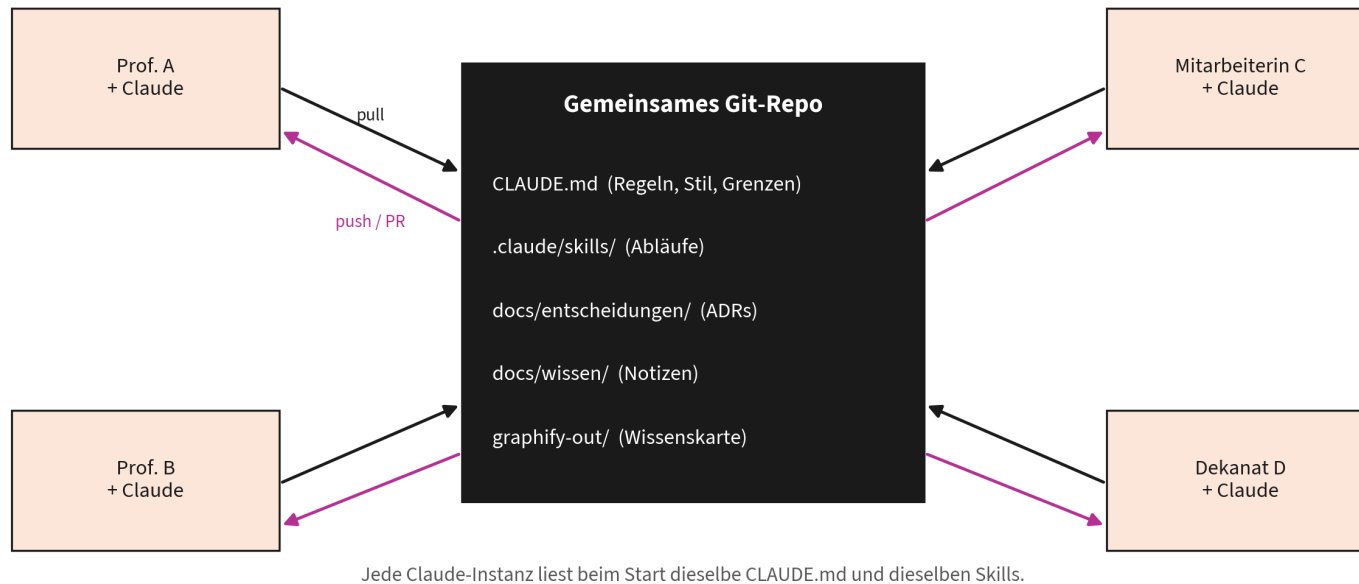
### Ein Gedächtnis für alle

- dieselbe CLAUDE.md, dieselben Skills
- Entscheidungen als ADR
- gemeinsame Leitplanken für alle

**Nicht die Instanzen miteinander reden lassen,** sondern ihnen dieselbe Wissensbasis geben.

Quellen: [5], [12], [13] (Verzeichnis im Anhang)

# Heute: das Repository als gemeinsames Gedächtnis



## ARBEITSABLAUF

1. git pull vor der Arbeit
1. eigener Zweig, eigene Arbeitskopie
1. Pull Request, ein Mensch prüft
1. Neues Wissen: docs, CLAUDE.md, Skill
1. Entscheidungen als ADR

**Vorlage im Handout:** zusammenarbeit/team-repo-vorlage mit CLAUDE.md, ADR-Skill und gemeinsamen Leitplanken.

Quellen: [14], [15] (Verzeichnis im Anhang)

# Parallel arbeiten, aber prüfbar

## IN EINER SITZUNG

### Unteragenten

- Spezialisten mit eigenem Kontext
- liefern nur das Ergebnis

## GLEICHZEITIG

### Worktrees

- parallele Sitzungen, je eigene Arbeitskopie
- Desktop automatisch, CLI --worktree

## ABGESTIMMT

### Agent Teams

- nur CLI, experimentell
- rund siebenfacher Verbrauch im Plan-Modus

## IM TEAM

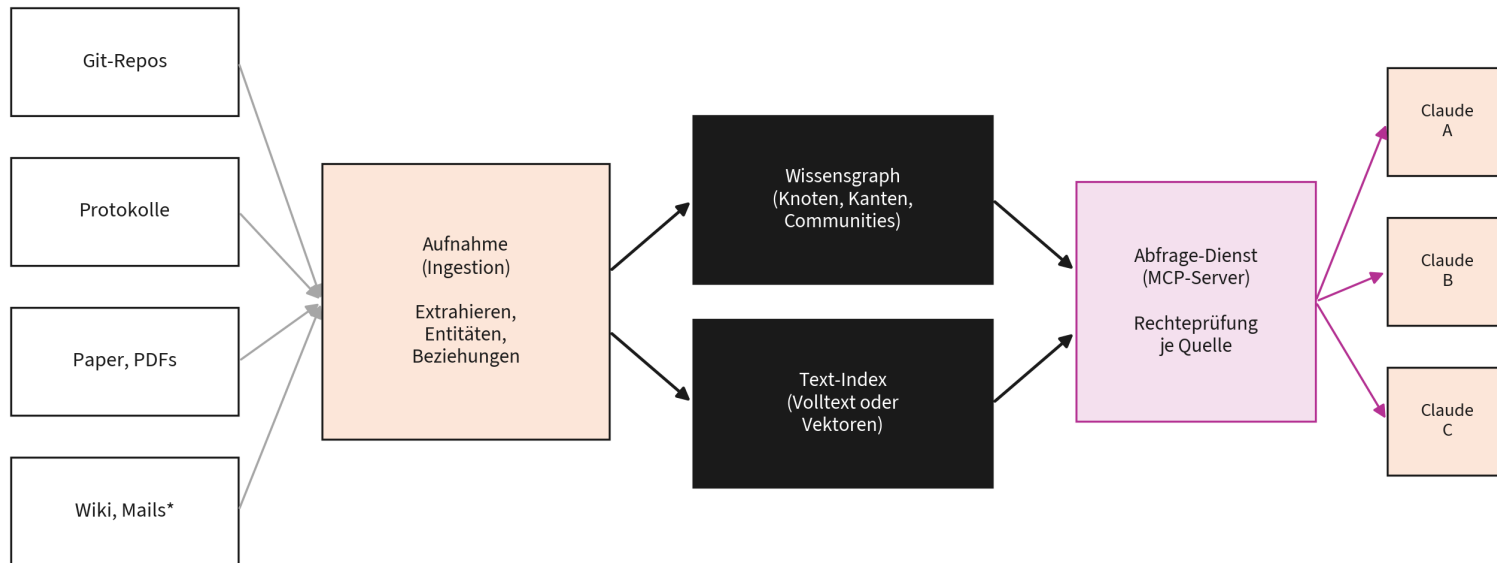
### GitHub-App

- @claude in Issues und PRs
- beachtet die CLAUDE.md

**Der Engpass ist die Prüfung,** nicht die Rechenleistung. Parallelisieren Sie nur, was unabhängig ist und was Sie noch prüfen können.

Quellen: [16], [17], [15], [18], [19] (Verzeichnis im Anhang)

# Morgen: ein gemeinsamer Wissensgraph



Aufwand: Aufbau, Pflege, Rechtekonzept, Qualitätssicherung der Extraktion. Datenschutz: \* nur mit Freigabe, Verarbeitung idealerweise in NRW (KI:connect) oder lokal.

## AUFWAND

### Ehrlich geschätzt

- Graphify im Repo: Stunden
- Team-MCP-Server: ein bis zwei Tage
- GraphRAG über viele Quellen: Wochen bis Monate, plus Pflege

**GraphRAG ist zuerst ein Governance-Projekt:** Zweck, Rechte pro Quelle, Löschkonzept, dann Technik.

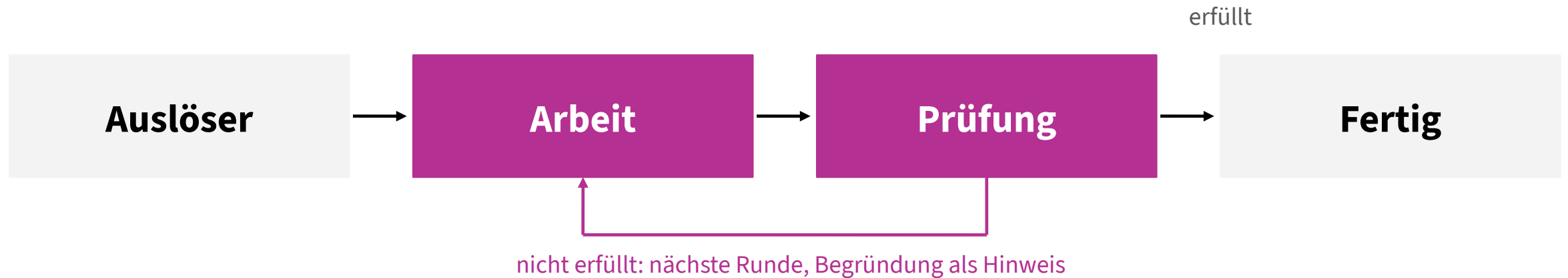
Quellen: [6], [10], [20] (Verzeichnis im Anhang)

# Wenn die KI sich selbst korrigiert

Ein Ausblick auf Aufgaben, die nicht nur erledigt, sondern auch geprüft und nachgebessert werden.



# Die nächste Stufe ist nicht der bessere Prompt, sondern der Regelkreis mit ehrlicher Prüfung.



Prompt Engineering: Wie frage ich? Context: Was weiß das Modell? Harness: Womit arbeitet es? Loop: Wer startet, prüft und stoppt?

Quellen: [21], [22], [23] (Verzeichnis im Anhang)

# Ein Loop läuft los

LIVE

ca. 2 Minuten

1. loop-demo zeigen: 14 Rückmeldungen, Prüfskript, leere Zusammenfassung
1. Prüfskript ausführen: nicht bestanden
1. Bedingung aus ziel.md mit /goal eingeben
1. laufen lassen; nach der Theorie schauen wir nach

## Falls es hakt

Musterlösung im Dozentenordner.  
Aufzeichnung vom Dienstag.

**Beobachtungsauftrag:** Wer entscheidet über „fertig“? Nicht das arbeitende Modell, sondern ein Prüfmodell und ein Skript.

Quellen: [24] (Verzeichnis im Anhang)

# Anatomie eines Loops: der Regelkreis

| Regelkreis  | Loop Engineering                  | Beispiel Dekanat                        |
|-------------|-----------------------------------|-----------------------------------------|
| Sollwert    | Ziel                              | alle Rückmeldungen drin, keine erfunden |
| Messglied   | Prüfung                           | Code vergleicht die IDs                 |
| Regler      | Entscheidungslogik                | vollständig? noch Versuche übrig?       |
| Stellglied  | Agent bzw. Modell                 | Claude schreibt die Zusammenfassung     |
| Rückführung | Hinweis an den nächsten Durchlauf | „Diese IDs fehlen: ...“                 |
| Gedächtnis  | Zustand auf der Platte            | Tabellen, Übergabedatei                 |

**Ein Regelkreis ist nur so gut wie sein Messglied.** Dazu kommen Trigger und eine Abbruchregel mit benannten Endzuständen.

Quellen: [25], [26], [23] (Verzeichnis im Anhang)

# Die Prüfung ist das Herzstück

**1 Deterministisch** Exit-Code, Referenz

**2 Regel** Schema, Linter, Richtlinie

**3 Verzögerte Realität** Betrieb, echte Rückmeldung

**4 Modell als Richter** Rubrik, zweites Modell

**5 Mensch** Freigabe, Stichprobe

## Selbstbestätigung

Wer sich selbst bewertet, wird besser bewertet, nicht besser.

## Specification Gaming

Test ändern statt Fehler beheben.

## Goodharts Gesetz

Wird die Kennzahl zum Ziel, taugt sie nicht mehr als Kennzahl.

Stufen 1 und 2: autonome Zone

**Macher und Prüfer trennen,** Prüfskript und Testdaten für den Agenten tabu.

Quellen: [23], [27], [28], [29], [30] (Verzeichnis im Anhang)

# Werkzeuge in Claude Code

| Werkzeug         | Nächster Lauf, wenn   | Stoppt, wenn                       | Einsatz                          |
|------------------|-----------------------|------------------------------------|----------------------------------|
| /goal            | voriger Lauf endet    | Prüfmodell: erfüllt oder unmöglich | prüfbarer Endzustand             |
| /loop            | Intervall abgelaufen  | Sie stoppen, Sitzung endet         | Kontrollen während der Arbeit    |
| Stop-Hook        | voriger Lauf endet    | Ihr Skript entscheidet             | dauerhafte Prüfungen             |
| Geplante Aufgabe | Zeitplan, Rechner an  | Aufgabe fertig                     | lokal, ab 1 Minute               |
| Routine (Cloud)  | Zeitplan, Rechner aus | Aufgabe fertig                     | ohne lokale Dateien, ab 1 Stunde |

**Bei /goal urteilt ein anderes Modell über „fertig“:** mit einem Prüfskript in der Bedingung sitzt die Prüfung auf Stufe 1.

Quellen: [24], [31], [13] (Verzeichnis im Anhang)

# Was der Loop gemacht hat

LIVE

ca. 2 Minuten

1. Verlauf: Entwurf, Prüfung, Nachbesserung
1. Prüfskript: bestanden, 14 von 14 IDs
1. `git diff --stat`: nur zusammenfassung.md geändert
1. Stoppgrund von `/goal` ansehen

## Falls es hakt

Musterlösung zeigen und prüfen.  
Aufzeichnung vom Dienstag.

**Beobachtungsauftrag:** Hat der Loop den Buchstaben der Prüfung erfüllt oder ihren Sinn? Lesen Sie die Zusammenfassung.

Quellen: [24] (Verzeichnis im Anhang)

# Wann welche Stufe?

|                                        |   |                                   |
|----------------------------------------|---|-----------------------------------|
| einmalige Aufgabe, sofort prüfbar      | → | einzelner Auftrag, Plan-Modus     |
| wiederkehrender Ablauf                 | → | Skill                             |
| viel Ausgabe, Spezialrolle             | → | Unteragent                        |
| mehrere unabhängige Aufgaben           | → | parallele Sitzungen mit Worktrees |
| prüfbarer Endzustand, mehrere Anläufe  | → | Loop mit /goal und Prüfskript     |
| regelmäßig wiederkehrend               | → | geplante Aufgabe oder Routine     |
| fester Prozess mit Formularen und Mail | → | n8n-Workflow mit Prüfschleife     |

**Goldene Regel:** Ein Loop lohnt sich nur, wenn das Ergebnis eines Durchlaufs die nächste Aktion verändert.

Quellen: [23], [32] (Verzeichnis im Anhang)

PAUSE

# Pause: 10 Minuten

## **Danach**

Block 8: was KI wirklich spart, zwölf Fälle aus Ihrem Haus, und Ihre nächsten Schritte.

## **Letzte Gelegenheit für Rückmeldungen**

Der QR-Code aus Folge 1 gilt weiter. Ich lese in der Pause mit; was offen bleibt, kommt in die Abschlussmail.

# Effizienz und nächste Schritte

Was die Evidenz sagt, wie Sie selbst messen, und was aus Ihren Rückmeldungen geworden ist.



# Wie viel Zeit spart KI wirklich?

**80 %**

## weniger Zeit pro Aufgabe

Anthropic: Schätzung aus 100.000 Claude-Gesprächen.  
Prüfzeit außerhalb des Chats ist nicht erfasst.

**19 %**

## mehr Zeit gebraucht

METR: randomisierte Studie mit 16 erfahrenen Entwicklern  
im eigenen Code, Werkzeugstand Anfang 2025.

**Effizienz entsteht bei häufigen, klar prüfbaren Aufgaben.** Messen Sie selbst.

Quellen: [33], [34] (Verzeichnis im Anhang)

# Warum die Zahlen so auseinanderliegen

1

**Aufgabentyp:** Schreiben und Strukturieren profitieren stark: etwa 40 % weniger Zeit bei Schreibaufgaben.

2

**Vorwissen:** Neulinge gewinnen mehr: im Kundenservice im Mittel rund 14 % mehr, bei Neulingen deutlich mehr.

3

**Prüfaufwand:** Jede KI-Ausgabe erzeugt eine Prüfpflicht.

4

**Wahrnehmung:** Gefühlt schneller ist nicht gemessen schneller.

5

**Unsichtbare Grenze:** Außerhalb der Fähigkeitsgrenze werden Lösungen häufiger falsch.

**Der größte Hebel ist selten „schneller schreiben“, sondern Arbeit, die gar nicht mehr anfällt.**

Quellen: [35], [36], [34], [37] (Verzeichnis im Anhang)

# Die Prozessinventur

Potenzial/Monat = Häufigkeit × Dauer  
× Automatisierbarkeit  
Netto Jahr 1 = 12 × Potenzial – Einrichtung  
– 12 × Pflege

- Häufigkeit und Dauer
- Anteil Muster statt Urteil (0 bis 1)
- Prüfbarkeit und Risiko
- Datenklasse nach der Ampel

## RECHENBEISPIEL DEKANAT

4 Termine × 60 Min. × 0,75

**180 Min. pro Monat**

abzüglich Einrichtung und Pflege:

**rund 24 Stunden im ersten Jahr**

**Oben stehen häufige, prüfbare, risikoarme Aufgaben mit grüner Datenklasse.** Der Skill /prozessinventur rechnet mit.

# Zwölf Fälle aus Ihrem Haus

## DEKANAT

### Serienbrief als PDF

Spart etwa

**4 Std./Monat**

Aufwand bis zum Betrieb

**0,5 Tage**

*läuft bereits*

## LEHRE

### Sprechstunden-Planer

Spart etwa

**8 Std./Monat**

Aufwand bis zum Betrieb

**3 Tage**

*noch nicht gebaut*

## LEHRE

### Aufgaben mit Prüfschleife

Spart etwa

**4 Std./Monat**

Aufwand bis zum Betrieb

**5 Tage**

*noch nicht gebaut*

## DEKANAT

### Posteingangs-Sortierer

Spart etwa

**20 Std./Monat**

Aufwand bis zum Betrieb

**5 Tage**

*noch nicht gebaut*

**Alle zwölf liegen ausgearbeitet bereit:** je mit Ablaufkette, Ampelfarbe, geschätztem Aufwand und der Stelle, an der ein Mensch freigibt. Suchen Sie sich einen aus, den Sie kennen, und wir gehen ihn gemeinsam durch.

Nutzen und Aufwand sind Schätzungen aus vergleichbaren Projekten, keine Messwerte. Gebaut und im Betrieb ist bisher nur der Serienbrief.

# Ihre Fragen

## WAS MICH MEISTENS GEFRAGT WIRD

### Vier davon nehme ich vorweg

- Womit fange ich am Montag an, wenn ich nur eine Stunde habe?
- Wie überzeuge ich Leitung oder Personalrat?
- Was kostet das alles, und wer bezahlt es?
- Was tue ich, wenn in einem Jahr alles anders heißt?

## LIEGT BEREIT, FALLS GEWÜNSCHT

### Folien im Anhang

- Graphify und Graft: die Befehle
- Rechtsrahmen kompakt
- Drei Muster: Claude und ein Ablaufwerkzeug
- Alles Weitere steht ausführlich im Skript

**Fragen Sie ruhig das Einfache:** Wenn Sie sich etwas fragen, fragen sich vier andere im Raum dasselbe. Was wir hier nicht schaffen, beantworte ich schriftlich.

# Ihre nächsten Schritte

## DIESE WOCHЕ

### Anfangen

- Einstellungen aus Kapitel 1 prüfen
- einen eigenen Skill anlegen

## DIESEN MONAT

### Bewerten

- Prozessinventur mit zehn Aufgaben
- THKI für eine interne Aufgabe testen

## DIESES SEMESTER

### Bauen

- einen Ablauf mit Prüfschleife
- ein Team-Repository für ein Projekt



Skript, Glossar, 41 Konzeptkarten,  
Kursempfehlungen, Beispielprojekte,  
Aufgabenkarten

**Unterlagen:** [github.com/ThorMaker/C3\\_thkoeln](https://github.com/ThorMaker/C3_thkoeln)

# Vielen Dank. Fragen?

Claude Code, Automatisierung, Wissen

Thorsten Fleuth | 18. September 2026

# Den n8n-Workflow einrichten

1. Data Tables termine und rueckmeldungen anlegen (Spalten laut README)
1. dekanat-live-klammer.json importieren
1. Zugangsdaten: eine für die KI (Basis-URL und Schlüssel), eine für Gmail, eine für den Kalender
1. Knoten Konfiguration anpassen: Modell, Mailadressen, Kalender
1. Workflow aktivieren, Formular-URLs kopieren
1. QR-Code für das Rückmeldeformular erzeugen
1. Testlauf, danach Testzeilen löschen

```
python3 tools/qr_erzeugen.py \  
<Production-URL>
```

## ANLEITUNG

### Im Handout-Repo

- n8n/README.md
- n8n/prompts/
- n8n/beispieldaten/

Quellen: [38], [39] (Verzeichnis im Anhang)

# Muster 2: ein Workflow als Werkzeug

```
claude mcp add --transport http n8n-dekanat <production-url> --header "Authorization: Bearer <token>"
```

## WEG A

### MCP Server Trigger

- ein Workflow stellt Werkzeuge bereit
- Test- und Produktions-URL
- Authentifizierung einschalten

## WEG B

### MCP-Zugang der Instanz

- eine Verbindung pro Instanz
- jeden Workflow einzeln freigeben
- ab 2.13 auch Workflows bauen

Quellen: [40], [41], [42] (Verzeichnis im Anhang)

# Drei Muster: Claude und ein Ablaufwerkzeug

## WORKFLOW STEUERT

### n8n ruft Claude

- KI ist ein Schritt im Ablauf
- unser Dekanats-Fall
- vorhersagbar und günstig

## AGENT STEUERT

### Claude ruft n8n

- Workflow als Werkzeug über MCP
- freigegebene Aktion statt Systemzugriff
- Brücke zu sicheren Agenten

## MENSCH MIT AGENT

### Claude baut n8n

- MCP-Zugang der Instanz, ab n8n 2.13
- n8n Skills für Claude Code
- Mensch prüft und veröffentlicht

**Muster 2 begrenzt Risiken:** Der Agent bekommt nicht das Buchungssystem, sondern einen geprüften Workflow, der genau eine Sache kann.

Quellen: [40], [41], [43] (Verzeichnis im Anhang)

# Modellwechsel in drei Feldern

| Was       | Wo es eingestellt wird           | Wechsel auf THKI bzw. KI:connect.nrw                 |
|-----------|----------------------------------|------------------------------------------------------|
| Basis-URL | Zugangsdatei des KI-Knotens      | https://chat.kiconnect.nrw/api/v1                    |
| Schlüssel | dieselbe Zugangsdatei            | in der THKI-Oberfläche selbst erzeugt                |
| Modell    | Knoten „Konfiguration“, ein Feld | laut Modellliste, z.B. inferenz-mistral-small-4-119b |

Der Workflow nutzt den KI-Knoten von n8n. Der spricht das OpenAI-Format, und jeder Anbieter mit OpenAI-kompatibler Schnittstelle lässt sich damit ansprechen, ohne einen einzigen Schritt im Ablauf zu ändern.

**Der Ablauf zählt, nicht das Modell:** Derselbe Workflow läuft auf NRW-Infrastruktur, wenn der Datenschutz es verlangt.

Quellen: [44], [45] (Verzeichnis im Anhang)

# Graphify und Graft: Befehle

## Graphify

```
uv tool install graphifyy  
graphify install  
  
# in Claude Code  
/graphify ./unterlagen  
/graphify ./unterlagen --update  
/graphify query "Frage"  
/graphify path "A" "B"  
/graphify explain "A"
```

## Graft

```
npm install -g @nanonets/graft  
graft init --dry-run  
graft init  
graft viz  
graft ask "Frage"  
graft build --deep  
graft telemetry disable  
graft uninstall
```

Quellen: [10], [11] (Verzeichnis im Anhang)

# Befehle auf einen Blick

```
/init /memory  
/clear /compact <Fokus>  
/rename /resume  
/model opusplan /effort  
/usage /context  
/plugin /mcp  
/goal <Bedingung> /goal clear  
/loop <Intervall> <Auftrag>  
/install-github-app
```

```
/handover (Skill im Handout)  
/prozessinventur (Skill im Handout)
```

```
claude -p "Auftrag"  
claude --worktree  
claude mcp add --transport http
```

```
python3 tools/qr_erzeugen.py <URL>  
python3 tools/prozessinventur.py
```

# Quellen

Webquellen abgerufen im September 2026. Vollständige Angaben im IEEE-Stil im Skript, Quellenverzeichnis je Kapitel.

- [1] J. Hoffmann et al.: Training Compute-Optimal Large Language Models. Advances in Neural Information Processing Systems 35 (NeurIPS 2022), 2022. <https://arxiv.org/abs/2203.15556>
- [2] N. F. Liu et al.: Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics, 2024. doi:10.1162/tacl\_a\_00638
- [3] P. Lewis et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020. <https://arxiv.org/abs/2005.11401>
- [4] A. Hogan et al.: Knowledge Graphs. ACM Computing Surveys, 2021. doi:10.1145/3447772
- [5] Anthropic: How Claude remembers your project. Claude Code Docs, o. J. <https://code.claude.com/docs/en/memory>
- [6] D. Edge et al.: From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv, 2024. <https://arxiv.org/abs/2404.16130>
- [7] Niklas Luhmann-Archiv: Zettelkasten. Universität Bielefeld, o. J. <https://niklas-luhmann-archiv.de/nachlass/zettelkasten>
- [8] N. Luhmann: Kommunikation mit Zettelkästen. Ein Erfahrungsbericht. Öffentliche Meinung und sozialer Wandel, 1981.
- [9] T. Forte: Building a Second Brain. 2022.
- [10] Graphify Labs: graphify: Turn any folder into a queryable knowledge graph. GitHub, o. J. <https://github.com/Graphify-Labs/graphify>
- [11] trailhq: Graft: Context layer for large codebases. GitHub, o. J. <https://github.com/trailhq/Graft>
- [12] Anthropic: Extend Claude with skills. Claude Code Docs, o. J. <https://code.claude.com/docs/en/skills>
- [13] Anthropic: Hooks reference. Claude Code Docs, o. J. <https://code.claude.com/docs/en/hooks>
- [14] M. Nygard: Documenting Architecture Decisions. Cognitect Blog, 2011. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [15] Anthropic: Desktop application. Claude Code Docs, o. J. <https://code.claude.com/docs/en/desktop>
- [16] Anthropic: Create custom subagents. Claude Code Docs, o. J. <https://code.claude.com/docs/en/sub-agents>
- [17] Anthropic: Agent teams. Claude Code Docs, o. J. <https://code.claude.com/docs/en/agent-teams>
- [18] Anthropic: Manage costs effectively. Claude Code Docs, o. J. <https://code.claude.com/docs/en/costs>
- [19] Anthropic: Claude Code GitHub Actions. Claude Code Docs, o. J. <https://code.claude.com/docs/en/github-actions>
- [20] TH Köln: Handreichung für Lehrende zum Umgang mit THKI Chat. 2025. <https://lehrpfade.th-koeln.de/thki-chat/>
- [21] IBM: What Is Loop Engineering? IBM Think, 2026. <https://www.ibm.com/think/topics/loop-engineering>
- [22] A. Osmani: Loop Engineering. addyosmani.com, 2026. <https://addyosmani.com/blog/loop-engineering/>

# Quellen (Fortsetzung)

Webquellen abgerufen im September 2026. Vollständige Angaben im IEEE-Stil im Skript, Quellenverzeichnis je Kapitel.

[23] S. Macedo: Stop Hand-Holding Your Coding Agent: Engineering the Loops that Replace Step-by-Step Prompting. arXiv, 2026. <https://arxiv.org/abs/2607.00038>

[24] Anthropic: Keep Claude working toward a goal. Claude Code Docs, o. J. <https://code.claude.com/docs/en/goal>

[25] N. Wiener: Cybernetics: Or Control and Communication in the Animal and the Machine. 1948.

[26] J. O. Kephart, D. M. Chess: The Vision of Autonomic Computing. Computer, 2003. doi:10.1109/MC.2003.1160055

[27] J. Pan et al.: Spontaneous Reward Hacking in Iterative Self-Refinement. arXiv, 2024. <https://arxiv.org/abs/2407.04549>

[28] A. Bondarenko et al.: Demonstrating Specification Gaming in Reasoning Models. arXiv, 2025. <https://arxiv.org/abs/2502.13295>

[29] M. Strathern: Improving ratings: audit in the British University system. European Review, 1997.

[30] J. Huang et al.: Large Language Models Cannot Self-Correct Reasoning Yet. International Conference on Learning Representations (ICLR), 2024. <https://arxiv.org/abs/2310.01798>

[31] Anthropic: Run prompts on a schedule. Claude Code Docs, o. J. <https://code.claude.com/docs/en/scheduled-tasks>

[32] E. Schluntz, B. Zhang: Building effective agents. Anthropic Research, 2024. <https://www.anthropic.com/research/building-effective-agents>

[33] A. Tamkin, P. McCrory: Estimating AI productivity gains from Claude conversations. Anthropic Research, 2025. <https://www.anthropic.com/research/estimating-productivity-gains>

[34] J. Becker et al.: Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. arXiv, 2025. <https://arxiv.org/abs/2507.09089>

[35] S. Noy, W. Zhang: Experimental evidence on the productivity effects of generative artificial intelligence. Science, 2023. doi:10.1126/science.adh2586

[36] E. Brynjolfsson, D. Li, L. Raymond: Generative AI at Work. 2023. doi:10.3386/w31161

[37] F. Dell'Acqua et al.: Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of AI on Knowledge Worker Productivity and Quality. 2023.

[38] n8n: Data Table node: Row operations. n8n Docs, o. J. <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.datatable/rows>

[39] n8n: n8n Documentation. n8n Docs, o. J. <https://docs.n8n.io>

[40] n8n: MCP Server Trigger. n8n Docs, o. J. <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-langchain.mcptrigger>

[41] n8n: Set up and use n8n MCP server. n8n Docs, o. J. <https://docs.n8n.io/advanced-ai/mcp/accessing-n8n-mcp-server/>

[42] Anthropic: Connect Claude Code to tools via MCP. Claude Code Docs, o. J. <https://code.claude.com/docs/en/mcp>

[43] n8n: Connect to n8n MCP server. n8n Docs, o. J. <https://docs.n8n.io/connect/connect-to-n8n-mcp-server>

[44] RWTH Aachen, IT Center: KI:connect. IT Center der RWTH Aachen, o. J. <https://www.itc.rwth-aachen.de/cms/it-center/services/kollaboration/~bndnjc/ki-connect/>

# Quellen (Fortsetzung)

Webquellen abgerufen im September 2026. Vollständige Angaben im IEEE-Stil im Skript, Quellenverzeichnis je Kapitel.

[45] FernUniversität in Hagen, ZLI: KI:connect.nrw. FernUniversität in Hagen, o. J. <https://www.fernuni-hagen.de/zli/produkte/weitere-tools/ki-connect-nrw.shtml>